SPI + I2C + UART: SHORT VIVA GUIDE

One demo, three protocols, two NUCLEO-F446RE boards, register-level C

Pitch | one-page sheets | comparison | 53 viva questions with short answers | numbers to memorize

YOUR 30-SECOND PITCH (say this when the examiner says 'explain your project')

I implemented the same switch demo three times on two NUCLEO-F446RE boards in pure bare-metal C, writing peripheral registers directly with no HAL: once with SPI, once with I2C and once with UART. Pressing the USER button on the master sends the byte 'S'. The slave blinks its LED 5 times and acknowledges with 'D'. The master then blinks twice to confirm. The goal was to compare how each protocol handles clocking, addressing, acknowledgement and waiting for a reply.

The common demo (identical in all three)

- Master board: button on PC13 (active low), LED PA5. Slave board: LED PA5.
- Request byte **'S'** = 0x53. Acknowledge byte **'D'** = 0x44.
- Slave: blink 5 x (200 ms ON + 200 ms OFF) = 2000 ms. Master success: blink 2 x (120 + 120) = 480 ms.
- Debounce 20 ms, poll button every 5 ms, delays are software loops, clock is reset-default HSI 16 MHz (SystemInit is an empty stub).

Wiring at a glance

```
SPI2  (PB13/14/15, AF5)   M --SCK---> S        3 signal wires + GND, STRAIGHT, no CS wire
                          M --MOSI--> S
                          M <--MISO-- S
I2C1  (PB8/PB9, AF4)      M <--SCL--> S        2 wires + GND, STRAIGHT, 4.7k pull-ups to 3.3 V
                          M <--SDA--> S
USART1 (PA9/PA10, AF7)    M TX -----> S RX     2 wires + GND, CROSSED (TX meets RX)
                          M RX <----- S TX
```

One-page sheets

SPI - Serial Peripheral Interface

In one line	Synchronous, full-duplex, master-clocked, push-pull. Every transfer swaps one byte each way.
Wires	SCK PB13, MISO PB14, MOSI PB15 (SPI2, AF5, APB1) + GND. No chip-select: both sides use software NSS.
Key settings	Master CR1 = 0x36C (MSTR, BR = /64, SSM, SSI, SPE). Slave CR1 = 0x240 (SSM, SPE). Mode 0 (CPOL = CPHA = 0), 8-bit, MSB first. 16 MHz / 64 = **250 kHz**; one byte = 32 us.
How the demo works	1) Master sends 'S' (gets junk 0x00 back). 2) Slave blinks 2 s, then loads 'D' into DR. 3) Master waits **blindly 2500 ms**. 4) Master sends dummy 0x00 and receives 'D'.
Core idea	Slave can never start. Its reply must be **preloaded before** the transfer, so the ack arrives **one transfer late** and the master needs a **dummy byte** to fetch it.
Watch out	No timeouts (infinite while loops), blind delay, slave deaf while blinking, flags: TXE (write) / RXNE (read).

I2C - Inter-Integrated Circuit

In one line	Synchronous, half-duplex, 2-wire, open-drain, addressed. Every byte is answered with ACK or NACK.
Wires	SCL PB8, SDA PB9 (I2C1, AF4, open-drain, APB1) + external 4.7k pull-ups to 3.3 V + GND.
Key settings	100 kHz: CR2 = 16 (MHz), CCR = 16 MHz / (2 x 100 kHz) = **80**, TRISE = 1000 ns / 62.5 ns + 1 = **17**. Slave address 0x28: OAR1 = **0x4050** (bit 14 = 1). Slave CR1 = 0x401 (ACK + PE). On the wire: 0x50 write, 0x51 read.
How the demo works	1) Master WRITE: START, 0x50, 'S', STOP. 2) Slave blinks 2 s, sets ack_ready. 3) Master waits **blindly 2500 ms**. 4) Master READ: START, 0x51, receives 'D', sends NACK, STOP.
Core idea	Address picks the slave; the **R/W bit announces direction first**, so the slave answers **in the same transaction**. No preloading or dummy byte. NACK from the master ends a read.
Watch out	ADDR is cleared by reading SR1 **then** SR2. Single-byte read: ACK = 0 **before** clearing ADDR, STOP right after. Master has guard counters (200000); slave loops have none.

UART - Universal Asynchronous Receiver/Transmitter

In one line	Asynchronous (no clock wire), full-duplex, point to point. Start bit, data bits, stop bit; both sides agree on baud.
Wires	PA9 TX, PA10 RX (USART1, AF7, APB2) + GND, **crossed**. RX has an internal pull-up (idle line is HIGH).
Key settings	BRR = **0x0683** ((104 << 4) | 3; USARTDIV = 16 MHz / (16 x 9600) = 104.17). CR1 = **0x200C** (UE, TE, RE). 8N1. Actual baud 9598 (-0.02%). One frame = 10 bits = 1.04 ms.
How the demo works	1) Master clears stale RX, sends 'S'. 2) Slave blinks 2 s, then **simply transmits** 'D'. 3) Master polls RXNE every 1 ms, **timeout about 6 s**. 4) Master reads 'D' and blinks twice.
Core idea	Either side can talk any time, so the ack is just another message. The master really waits for the reply instead of guessing.
Watch out	TX-to-TX mistake, baud mismatch, error flags (FE, NF, ORE) never checked, slave deaf while blinking.

Comparison table (memorize this)

Aspect	SPI	I2C	UART
Clock	Synchronous, master SCK	Synchronous, master SCL (slave may stretch)	Asynchronous, agreed baud
Wires used here	3 + GND (no CS)	2 + GND + 2 pull-ups	2 + GND
Wiring	Straight	Straight	Crossed (TX to RX)
Output stage	Push-pull	Open-drain + pull-ups	Push-pull
Duplex	Full	Half	Full
Selecting a slave	CS line (software NSS here)	7-bit address in frame	None (point to point)
Who may start	Master only	Master only	Either side
Speed here	250 kHz (can reach MHz)	100 kHz (400 kHz fast mode)	9600 baud
Acknowledge	None built in	ACK/NACK bit per byte	None; 'D' byte is the ack
Reply timing	Preloaded, one transfer late, dummy byte	On demand, same transaction	Slave transmits when ready
Master waits	Blind 2500 ms	Blind 2500 ms	Polls RXNE, 6 s timeout
Hang protection	None	Guard counters on master	Software timeout on master
Bus / clock	APB1	APB1	APB2
Typical use	Fast chips: flash, display, ADC	Many slow sensors on 2 wires	Console, GPS, Bluetooth, PC link

Numbers to memorize

Item	SPI	I2C	UART
Peripheral / pins	SPI2: PB13, PB14, PB15	I2C1: PB8 SCL, PB9 SDA	USART1: PA9 TX, PA10 RX
Alternate function	AF5	AF4	AF7
Clock enable	APB1ENR bit 14	APB1ENR bit 21	APB2ENR bit 4
Config values	CR1 0x36C / 0x240	CR2 16, CCR 80, TRISE 17, OAR1 0x4050, CR1 0x401	BRR 0x683, CR1 0x200C
Data flags	SR: bit 1 TXE, bit 0 RXNE	SR1: 0 SB, 1 ADDR, 2 BTF, 4 STOPF, 6 RXNE, 7 TXE, 10 AF	SR: bit 7 TXE, bit 5 RXNE
Speed math	16 MHz / 64 = 250 kHz	16 MHz / (2 x 80) = 100 kHz	16 MHz / 1667 = 9598 baud
Byte time	32 us	90 us (9 clocks)	1.04 ms (10 bits)

Viva questions with short answers

A. General project questions

Q1. Explain your project in 30 seconds.

A. See the pitch on page 1: the same button, blink and acknowledge demo in SPI, I2C and UART, bare-metal on two NUCLEO-F446RE boards.

Q2. Why bare-metal registers instead of HAL?

A. To understand exactly what the hardware does: which clock, pin mode, alternate function and flags are involved. HAL hides all of that and adds code size.

Q3. Which microcontroller and clock?

A. STM32F446RE (Cortex-M4F). Everything runs at reset-default HSI 16 MHz because SystemInit is an empty stub.

Q4. Steps to use any peripheral at register level?

A. Enable clock, set pin mode, select alternate function, set pulls (and open-drain), configure the peripheral, enable it last, then use it by polling flags.

Q5. Why volatile on every register macro?

A. Registers change without the program touching them. volatile forces every read and write, so polling loops and delay loops are not optimized away.

Q6. Why clear-then-set (&= ~mask; |= value)?

A. It changes only the chosen bit field and leaves neighbouring pins untouched.

Q7. Why empty weak SystemInit and __libc_init_array?

A. The startup file calls them. Empty weak stubs satisfy the linker without CMSIS or newlib setup and can be overridden.

Q8. How is the button read reliably?

A. PC13 is active low with a pull-up. The code detects a new press edge, waits 20 ms to debounce and re-checks.

Q9. How do delays work and what is the drawback?

A. A software busy-wait loop (about 1300 iterations per ms). It depends on clock and optimization, and blocks the CPU.

B. SPI questions

Q10. What are the four SPI signals?

A. SCK, MOSI, MISO and CS/NSS. This demo uses software NSS, so only three signal wires plus GND.

Q11. Which SPI mode and frame?

A. Mode 0 (CPOL 0, CPHA 0), 8-bit, MSB first.

Q12. How is 250 kHz obtained?

A. SPI2 is on APB1 at 16 MHz. BR = 101 divides by 64: 16 MHz / 64 = 250 kHz.

Q13. Why does the master send a dummy byte?

A. SPI is a shift-register swap. The slave can only shift data out while the master clocks, so reading requires writing.

Q14. Why does the ack arrive one transfer late?

A. The slave's reply must be in DR before the transfer starts. 'D' is loaded after the blink, so it leaves during the next transfer.

Q15. What are SSM and SSI?

A. SSM = 1 ignores the NSS pin and uses SSI as its level. Slave SSI = 0 means always selected; master SSI = 1 avoids a mode fault.

Q16. TXE and RXNE: when do you use each?

A. Write DR only when TXE = 1; read DR only when RXNE = 1 (which clears it).

Q17. Why a pull-down on MISO?

A. A defined 0 if the slave is missing or not driving, instead of a floating input.

Q18. Can an SPI slave start communication?

A. No. Only the master generates the clock.

C. I2C questions

Q19. Why open-drain with pull-ups?

A. Any device can pull low, nobody drives high, so no short circuit. The resistor makes the line high. It also enables clock stretching and multi-master.

Q20. How is 100 kHz produced?

A. CCR = fPCLK1 / (2 x fSCL) = 16 MHz / 200 kHz = 80, so high time = low time = 5 us.

Q21. What are CR2 = 16 and TRISE = 17?

A. FREQ = APB1 clock in MHz. TRISE = max rise time / TPCLK1 + 1 = 1000 ns / 62.5 ns + 1 = 17.

Q22. What is on the wire for address 0x28?

A. 0x50 for write and 0x51 for read: the address shifted left, with the R/W bit in bit 0.

Q23. Define START and STOP.

A. START: SDA falls while SCL is high. STOP: SDA rises while SCL is high.

Q24. How is ADDR cleared?

A. Read SR1, then read SR2. In the slave the polling loop already read SR1.

Q25. Why does the master NACK the last byte it reads?

A. It tells the slave to stop sending so the master can issue STOP. For one byte: clear ACK before clearing ADDR, then STOP immediately.

Q26. How does the slave know to transmit or receive?

A. SR2.TRA: 0 = receiver (master writes), 1 = transmitter (master reads).

Q27. What if the slave is missing?

A. The address gets no ACK so AF sets. The master clears AF, sends STOP and returns 0; guard counters prevent hangs.

D. UART questions

Q28. Why is UART asynchronous?

A. No clock wire. Each side has its own baud clock set to the same value, and each start bit re-synchronizes the receiver.

Q29. Draw the frame for 'S'.

A. Idle high, start 0, data LSB first 1 1 0 0 1 0 1 0, stop 1: 10 bits in total.

Q30. How is BRR = 0x0683 calculated?

A. USARTDIV = 16 MHz / (16 x 9600) = 104.1667. Mantissa 104 = 0x68, fraction 0.1667 x 16 = about 3. BRR = (104 << 4) | 3.

Q31. Why are TX and RX crossed?

A. A transmitter must feed the other device's receiver. TX to TX would leave both receivers unconnected.

Q32. Why a pull-up on RX?

A. The line idles high; a floating RX could show noise that looks like start bits.

Q33. TXE vs TC?

A. TXE: data register free, you may write the next byte. TC: the last frame has completely left the shifter.

Q34. Which errors can UART detect?

A. Framing (FE), noise (NF), overrun (ORE) and parity (PE) if enabled. This code checks none of them.

Q35. Is UART master/slave?

A. No. It is peer to peer; both can transmit at any time. The roles here are an application choice.

Q36. How does the master avoid hanging?

A. It polls RXNE in a loop of 6000 x 1 ms and gives up after about 6 s.

E. Comparison and choice questions

Q37. Which protocol is fastest? Fewest wires? Most devices per wire pair?

A. SPI is fastest. UART and I2C both need 2 wires. I2C supports many devices on the same 2 wires.

Q38. Which protocol has a built-in acknowledge?

A. I2C (ACK/NACK each byte). SPI and UART have none, so the demo invents an ack byte 'D'.

Q39. Which is full duplex?

A. SPI and UART. I2C is half duplex (one SDA line).

Q40. How does the slave's reply differ in each?

A. SPI: preloaded, one transfer late. I2C: on demand after the R/W bit. UART: transmitted whenever ready.

Q41. When would you choose UART over SPI?

A. For simple point-to-point links over longer distances or to a PC/module, where a shared clock wire is inconvenient.

Q42. Why does I2C need pull-ups but SPI does not?

A. I2C lines are open-drain (shared, wired-AND); SPI lines are push-pull and each driven by one device.

Q43. Why are SPI2/I2C1 on APB1 but USART1 on APB2?

A. That is how the chip's bus matrix places them; APB2 is the faster bus. The clock-enable bit lives in APB1ENR or APB2ENR accordingly.

F. Critical thinking and improvements

Q44. What are the main weaknesses of your code?

A. Blind 2500 ms delays (SPI, I2C), no timeouts in SPI and the I2C slave, blocking blink, polling everywhere, unchecked error flags (UART).

Q45. How would you replace the blind delay?

A. Poll for the reply: dummy-byte polling or a READY pin (SPI), a status byte (I2C), RXNE with timeout (already in UART).

Q46. How would you improve it?

A. Use interrupts or DMA, a timer (SysTick) for timeouts and delays, non-blocking blink, error checks with retry, hardware NSS for SPI.

Q47. What if the slave blinked 3 s instead of 2 s?

A. SPI: master reads 0x00, no confirmation. I2C: read likely times out. UART: still works (6 s timeout).

Q48. What if the clock changed from 16 MHz?

A. SPI/I2C speeds change and delays become wrong; I2C CR2/CCR/TRISE and UART BRR would need recalculating.

G. Tricky cross-questions

Q49. Why is the I2C slave not preloaded like the SPI slave?

A. Because the R/W bit tells it about the read first; it writes DR during the transaction, with SCL stretched if needed.

Q50. What happens if the I2C pull-ups are removed?

A. Lines cannot rise (open-drain), so SCL/SDA stay low or float, START/ADDR never complete and the master's guard counters expire.

Q51. What if UART baud differs by 5%?

A. Sampling drifts across the 10-bit frame, causing framing errors and wrong bytes. The tolerance is only a few percent.

Q52. In SPI, what does the slave send back during the first transfer?

A. Whatever was preloaded (0x00). 'D' comes only on the second transfer.

Q53. What does reading DR do in each protocol?

A. SPI and UART: reading DR clears RXNE. I2C: reading DR clears RXNE; ADDR needs SR1 then SR2, STOPF needs SR1 then a CR1 write.

> **LAST-MINUTE VIVA TIPS**
>
> 1) Start with the pitch, then offer: 'Would you like SPI, I2C or UART first?'. 2) Always link a register line to a reason (clock, pin, AF, pull, enable). 3) Admit weaknesses first and give the fix: examiners reward that. 4) The three key ideas: SPI = swap and preload, I2C = address plus ACK, UART = no clock plus start/stop bits.